FFmpeg 4.2.2 MinGW 32-bit 重编译指南

> **重要更新（2026-07-29）**：经过 H264 视频崩溃修复过程中的深入验证，本文档最初关于"FFmpeg DLL 是 MSVC 编译导致 ABI 不兼容"的假设**已被推翻。**
> FFmpeg DLLs 实际是 MinGW 编译的 Zeranoe 构建，`sws_scale` 在正确使用 `.dll.a` 导入库后可以正常工作。本指南中的重编译步骤作为**备用方案**保留，仅在确实遇到 ABI 问题时参考。

背景

⚠️ 假设更新说明（2026-07-29）

原假设（已被推翻）：项目使用 MinGW 32-bit 编译器（Qt 5.12.11 MinGW 32-bit），但 FFmpeg DLL 是用 MSVC 编译的，两者存在 ABI 不兼容，导致 `sws_scale` 函数内部 SSE 指令因栈对齐差异触发 SIGSEGV 崩溃。

验证结论（最新）：经过对 `ffmpeg\bin\avcodec-58.dll` 等动态库的依赖分析：

```
ffmpeg\bin\avcodec-58.dll
  └── 依赖: msvcrt.dll (非 msvcrXXX.dll)
  └── 结论: MinGW 编译的 Zeranoe 构建，非 MSVC 编译
```

- **MSVC 编译的 DLL** 会依赖 `vcruntime140.dll`、`MSVCR120.dll` 等 MSVC 运行时库
- **MinGW 编译的 DLL** 依赖 `msvcrt.dll`（Windows 系统 C 运行时，MSVC 和 MinGW 都可使用）
- 项目使用的 FFmpeg DLLs 依赖 `msvcrt.dll`，**确认为 MinGW 编译的 Zeranoe 构建**

关键结论：

1. FFmpeg DLLs 与 MinGW 32-bit 代码**ABI 完全兼容**，无需重编译
2. `sws_scale` 函数可以安全使用，不会因栈对齐问题崩溃
3. 之前 H264 编码崩溃的真正原因是 **QImage 隐式共享在多线程环境下的竞态条件**（详见 H264视频崩溃修复报告.md）
4. 导入库格式从 `.lib` 切换到 `.dll.a` 是必要的（防止潜在符号修饰问题），但这不是崩溃原因

实际采用的修复方案（无需重编译 FFmpeg）

在确认 FFmpeg DLLs 是 MinGW 编译后，实际修复仅涉及以下两点：

修复 1：导入库格式修正（`.lib` → `.dll.a`）

项目原使用 MSVC 格式的 `.lib` 导入库，应使用 MinGW 格式的 `.dll.a`：

```
# 修改前（MSVC 格式）
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.lib \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.lib \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.lib

# 修改后（MinGW 标准格式）
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.dll.a
```

导入库格式	编译器	说明
`.lib`	MSVC	COFF 格式，符号修饰与 MinGW 不完全匹配
`.dll.a`	MinGW	标准 GNU 格式，符号修饰完全匹配

修复 2：sws_scale 替代手动 RGB→YUV 转换

由于确认 DLLs 是 MinGW 编译的，`sws_scale` 可以安全使用：

```
// 使用 FFmpeg sws_scale 替代手动 BT.601 转换
m_swsCtx = sws_getContext(width, height, AV_PIX_FMT_RGB24,
                          width, height, AV_PIX_FMT_YUV420P,
                          SWS_BILINEAR, nullptr, nullptr, nullptr);

const uint8_t *srcData[1] = { rgbImg.constBits() };
int srcLinesize[1] = { rgbImg.bytesPerLine() };
sws_scale(m_swsCtx, srcData, srcLinesize, 0, m_height,
          m_frame->data, m_frame->linesize);
```

历史假设（仅供记录，已推翻）

以下是最初的假设和诊断依据，现已确认不成立：

- ~~`.lib` 文件为 MSVC 格式（无 `longnames` 表）~~ — `.lib` 格式仅说明导入库是 MSVC 格式，不代表 DLL 本身是 MSVC 编译的
- ~~DLL 链接 `msvcrt.dll` 无法区分编译器~~ — 实际上 MSVC 编译会链接 `vcruntimeXXX.dll`，而 MinGW 编译链接 `msvcrt.dll`，可以区分

- ~~`swscale-5.dll` 中存在 SSE 指令与 MinGW 32-bit 的 4 字节默认栈对齐不兼容~~ — MinGW 编译的 DLL 内部已正确处理栈对齐
- 项目 `lib/` 目录下已存在 MinGW 格式的 `.dll.a` 文件（`libavcodec.dll.a` 等），但未被使用 — 这一点是真实的，需要切换

备用方案：使用 MSYS2 + MinGW 32-bit 重编译 FFmpeg

> 以下重编译步骤作为**备用方案**保留。在当前项目中**不需要执行**，因为已确认 FFmpeg DLLs 是 MinGW 编译的。仅在未来遇到真实 ABI 问题、或需要自定义编解码器配置时参考。

第一阶段：安装 MSYS2 环境

步骤 1.1：下载并安装 MSYS2

操作事项：下载 MSYS2 安装包并安装

具体方法：

1. 打开浏览器，访问 `https://www.msys2.org/`
2. 下载最新版 MSYS2 安装包（文件名类似 `msys2-x86_64-20240113.exe`）
3. 双击运行安装程序
4. 安装路径使用默认的 `C:\msys2`（路径中不要有中文或空格）
5. 安装完成后，勾选"立即运行 MSYS2"

所需工具/权限：浏览器、管理员权限

预期结果：MSYS2 终端窗口打开，显示类似 `user@DESKTOP MINGW64 ~` 的提示符

步骤 1.2：更新 MSYS2 包管理器

操作事项：在 MSYS2 终端中更新系统包

具体方法：在打开的 MSYS2 MSYS 终端中依次执行：

```
# 更新包数据库和核心包（可能需要关闭终端重新打开）
pacman -Syu

# 关闭终端后重新打开 MSYS2 MSYS 终端，继续更新剩余包
pacman -Su
```

注意：执行 `pacman -Syu` 时可能提示关闭终端，这是正常的。关闭后从开始菜单重新打开 MSYS2 MSYS 终端继续执行。

预期结果：所有包更新完成，显示 `there is nothing to do` 或类似提示

步骤 1.3：安装 MinGW 32-bit 工具链

操作事项：安装 32 位 GCC 编译器和相关构建工具

具体方法：在 MSYS2 MSYS 终端中执行：

```
# 安装 MinGW 32-bit GCC 编译器
pacman -S mingw-w64-i686-gcc

# 安装构建工具
pacman -S mingw-w64-i686-pkg-config

# 安装 NASM 汇编器
pacman -S mingw-w64-i686-nasm

# 安装 make 工具
pacman -S make
```

每次执行后按 Y 确认安装。

预期结果：安装完成后，在 **MSYS2 MinGW 32-bit** 终端中验证：

```
gcc --version
# 应输出 i686-w64-mingw32-gcc 版本信息

nasm --version
# 应输出 NASM 版本信息

make --version
# 应输出 GNU Make 版本信息
```

关键区分：

终端名称	提示符	用途
MSYS2 MSYS	`MSYS ~`	安装软件包（pacman）
MSYS2 MinGW 32-bit	`MINGW32 ~`	编译 32 位程序（gcc, make）
MSYS2 MinGW 64-bit	`MINGW64 ~`	编译 64 位程序（不使用）

第二阶段：编译 FFmpeg 4.2.2

步骤 2.1：下载 FFmpeg 4.2.2 源码

操作事项：下载与当前项目版本一致的 FFmpeg 源码

具体方法：在 MSYS2 MinGW 32-bit 终端中执行：

```
# 进入用户主目录
cd ~

# 下载 FFmpeg 4.2.2 源码包
wget https://ffmpeg.org/releases/ffmpeg-4.2.2.tar.bz2

# 解压
tar xjf ffmpeg-4.2.2.tar.bz2

# 进入源码目录
cd ffmpeg-4.2.2
```

预期结果：目录中包含 `configure`、`Makefile`、`libavcodec`、`libswscale` 等文件和目录

步骤 2.2：安装 libx264 编码器支持（可选）

操作事项：安装 x264 软编码器库

具体方法：在 MSYS2 MSYS 终端中执行：

```
pacman -S mingw-w64-i686-x264
```

说明：项目当前实际使用 libx264 编码器，建议安装。如果不需要 libx264，可跳过。

步骤 2.3：配置 FFmpeg 编译选项

操作事项：运行 configure 脚本，设置编译参数

重要：必须在 **MSYS2 MinGW 32-bit** 终端中执行（提示符显示 `MINGW32 ~`）。

具体方法：

```
cd ~/ffmpeg-4.2.2

# 创建安装目录
mkdir -p ~/ffmpeg-4.2.2-mingw32
```

```
# 配置编译选项
./configure \
    --prefix=$HOME/ffmpeg-4.2.2-mingw32 \
    --arch=x86 \
    --target-os=mingw32 \
    --extra-cflags="-std=gnu11" \
    --enable-shared \
    --disable-static \
    --disable-asm \
    --disable-yasm \
    --disable-doc \
    --disable-programs \
    --disable-network \
    --disable-everything \
    --enable-encoder=libx264 \
    --enable-decoder=h264 \
    --enable-muxer=avi \
    --enable-demuxer=h264 \
    --enable-protocol=file \
    --enable-swscale \
    --enable-avcodec \
    --enable-avformat \
    --enable-avutil \
    --enable-swresample \
    --enable-postproc \
    --enable-gpl \
    --enable-libx264
```

参数说明:

参数	含义
`--prefix`	安装路径
`--arch=x86`	32 位架构
`--target-os=mingw32`	目标操作系统
`--extra-cflags="-std=gnu11"`	强制 C11 标准，兼容 GCC 16
`--enable-shared`	编译动态库（DLL）
`--disable-static`	不编译静态库
`--disable-asm`	禁用汇编优化，避免 SSE 对齐问题
`--disable-yasm`	禁用 YASM 汇编器
`--disable-doc`	不编译文档
`--disable-programs`	不编译 ffmpeg/ffprobe 等工具

参数	含义
`--enable-swscale`	启用 swscale 库
`--enable-gpl`	启用 GPL 许可（libx264 需要）

预期结果：configure 完成后显示配置摘要，包含：

```
Enabled encoders:  libx264
Enabled decoders:  h264
Enabled swscale:   yes
```

步骤 2.4：编译 FFmpeg

操作事项：执行 make 编译

具体方法：在同一个 MINGW32 终端中执行：

```
cd ~/ffmpeg-4.2.2
make -j8
```

编译时间：约 5-15 分钟，取决于电脑性能

预期结果：编译完成无报错

步骤 2.5：安装编译结果

操作事项：将编译好的文件安装到指定目录

具体方法：

```
cd ~/ffmpeg-4.2.2
make install

# 查看安装结果
ls ~/ffmpeg-4.2.2-mingw32/bin/
ls ~/ffmpeg-4.2.2-mingw32/lib/
```

预期结果：安装目录结构：

```
~/ffmpeg-4.2.2-mingw32/
├── bin/                ← DLL 文件
│   ├── avcodec-58.dll
│   ├── avutil-56.dll
│   ├── swscale-5.dll
```

```
│   └── ...
├── include/              ← 头文件（与原项目相同，无需替换）
└── lib/                  ← 链接库文件
    ├── libavcodec.dll.a
    ├── libswscale.dll.a
    └── ...
```

步骤 2.6：验证编译产物

操作事项：确认 DLL 是 MinGW 编译的

具体方法：

```
# 检查 swscale DLL 的依赖（应显示 msvcrt，不显示 vcruntime）
objdump -p ~/ffmpeg-4.2.2-mingw32/bin/swscale-5.dll | grep "DLL Name"
```

预期结果：输出中包含 `msvcrt.dll`，不包含 `vcruntime140.dll` 或 `MSVCR120.dll`

第三阶段：替换项目文件

步骤 3.1：备份原 FFmpeg 文件

操作事项：备份项目中当前的 FFmpeg 文件

具体方法：

1. 打开 `WeChat\`
2. 右键 `ffmpeg-4.2.2` 文件夹 → 复制 → 粘贴在同一目录
3. 重命名为 `ffmpeg-4.2.2-backup-msvc`

步骤 3.2：复制新的 DLL 文件

操作事项：用 MinGW 编译的 DLL 替换 MSVC 编译的 DLL

具体方法：

1. 从 MSYS2 安装目录的编译结果中复制 DLL：

   ```
   C:\msys64\home\<用户名>\ffmpeg-4.2.2-mingw32\bin\
   ```

   或自定义安装路径下的对应目录
2. 复制以下 8 个 DLL 文件：

- avcodec-58.dll
- avdevice-58.dll
- avfilter-7.dll
- avformat-58.dll
- avutil-56.dll
- postproc-55.dll
- swresample-3.dll
- swscale-5.dll

3. 粘贴到项目的 FFmpeg bin 目录，**覆盖**原有文件：

```
WeChat\ffmpeg-4.2.2\bin\
```

4. 同时粘贴到 release 目录，**覆盖**原有文件：

```
build-WeChat-Desktop_Qt_5_12_11_MinGW_32_bit-release\release\
```

步骤 3.3：复制新的链接库文件

操作事项：用 MinGW 格式的 `.dll.a` 文件替换

具体方法：

1. 从编译结果的 lib 目录复制 `lib*.dll.a` 文件
2. 粘贴到项目的 FFmpeg lib 目录，**覆盖**已有的同名文件：

```
WeChat\ffmpeg-4.2.2\lib\
```

第四阶段：修改项目配置

步骤 4.1：修改 vcc-client.pro 链接库路径

操作事项：将 `.lib` 引用改为 MinGW `.dll.a` 格式

具体方法：打开 `vcc-client.pro`，将 LIBS 配置修改为直接引用 `.dll.a` 文件：

```
# [修复] 改用 MinGW 编译的 .dll.a 格式导入库
# 注意：直接引用 .dll.a 完整路径，而非 -L/-l 简写形式
# 这样更明确地指定使用 MinGW 格式导入库
LIBS += $$PWD/ffmpeg-4.2.2/lib/libavcodec.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavdevice.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavfilter.dll.a \
```

```
        $$PWD/ffmpeg-4.2.2/lib/libavformat.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libavutil.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libpostproc.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswresample.dll.a \
        $$PWD/ffmpeg-4.2.2/lib/libswscale.dll.a

LIBS += $$PWD/SDL2-2.0.10/lib/x86/SDL2.lib
```

注意：`videorender.pri` 中的 FFmpeg 库引用也需同步修改为 `.dll.a` 格式。

第五阶段：重新编译项目并测试

步骤 5.1：清理并重新编译项目

操作事项：在 Qt Creator 中清理旧的编译产物并重新构建

具体方法：

1. 打开 Qt Creator → 打开项目 `vcc-client.pro`
2. 确认构建套件为 `Desktop Qt 5.12.11 MinGW 32-bit Release`
3. 菜单 → **构建** → **清理所有项目**
4. 菜单 → **构建** → **重新构建项目 "WeChat"**

预期结果：编译无错误，在 release 目录生成新的 `WeChat.exe`

步骤 5.2：运行测试

操作事项：运行程序，验证视频功能

具体方法：运行 `WeChat.exe`，登录账号，进入房间，开启摄像头。

关键日志确认点：

```
H264Encoder: using encoder libx264 (verified by avcodec_open2)
H264Encoder: init success, 640 x 480 @ 30 fps
slot_sendVideoFrame called, img size: 640 x 480 , m_id: 3   ← 持续运行不崩溃
[slot_sendVideoFrame] H264 encode: ba.size= XXXXX totalFragments= X
[slot_SendVideo] SendData OK, packType= 10017
```

预期结果：程序不崩溃，视频画面正常显示，`sws_scale` 正常工作（已验证 MinGW 编译的 DLL 与 MinGW 代码 ABI 兼容）。

已知问题与解决方案

问题 1：GCC 版本过新导致 configure 失败

现象：`gcc is unable to create an executable file`，config.log 显示 `C compiler test failed`

原因：MSYS2 安装的 GCC 版本过新（如 GCC 16），默认使用 C23 标准，与 FFmpeg 4.2.2 不兼容

解决：configure 时添加 `--extra-cflags="-std=gnu11"` 强制使用 C11 标准

问题 2：在错误的终端中执行命令

现象：`gcc: command not found` 或 `pkg-config not found`

原因：在 MSYS2 MSYS 终端中执行了编译命令

解决：必须使用 **MSYS2 MinGW 32-bit** 终端（提示符显示 `MINGW32 ~`）

问题 3：libx264 未找到

现象：configure 报错 `libx264 not found`

解决：在 MSYS2 MSYS 终端中执行 `pacman -S mingw-w64-i686-x264`

操作清单

序号	操作	终端类型	耗时
1.1	安装 MSYS2	Windows	3 分钟
1.2	更新 MSYS2 包	MSYS	2 分钟
1.3	安装 MinGW 32-bit 工具链	MSYS	2 分钟
2.1	下载 FFmpeg 4.2.2 源码	MINGW32	1 分钟
2.2	安装 libx264（可选）	MSYS	1 分钟
2.3	配置 FFmpeg	MINGW32	1 分钟
2.4	编译 FFmpeg	MINGW32	5-15 分钟
2.5	安装 FFmpeg	MINGW32	1 分钟

序号	操作	终端类型	耗时
3.1	备份原 FFmpeg 文件	Windows	1 分钟
3.2	替换 DLL 文件	Windows	1 分钟
3.3	替换 .dll.a 文件	Windows	1 分钟
4.1	修改 vcc-client.pro	Qt Creator	2 分钟
5.1	清理并重新编译项目	Qt Creator	3-5 分钟
5.2	运行测试	Qt Creator	2 分钟